
Football Possession Intelligence API

Predicción de la probabilidad de disparo
a partir de secuencias de posesión en fútbol
mediante aprendizaje profundo

PR-AUC (prueba)
0.870

Posesiones
21,080

Partidos analizados
127

Modelo
GRU

Competencias
4 ligas

Mejora vs. baseline
+30.7 pp

Maestría en Ciencia de Datos

Curso: Aprendizaje Profundo

Junio 2026

Resumen

Este reporte documenta el desarrollo de un sistema de aprendizaje profundo para estimar la probabilidad de que una posesión ofensiva en fútbol termine en disparo: $P(\text{disparo} \mid \text{posesión})$. El sistema procesa secuencias de eventos de StatsBomb Open Data correspondientes a 127 partidos de cuatro competencias europeas (Bundesliga 2023/24, La Liga 2020/21 y Ligue 1 2021/22 + 2022/23), produciendo un total de 21,080 posesiones etiquetadas.

El modelo principal es una red recurrente tipo GRU (Gated Recurrent Unit) de dos capas que opera sobre secuencias de eventos codificados con 40 características por paso temporal. Se compara contra un modelo de regresión logística entrenado sobre características agregadas por posesión.

El GRU alcanza un **PR-AUC de 0.852 en validación y 0.870 en prueba**, superando al baseline en +27 puntos porcentuales. Los resultados se obtienen sin fuga de datos entre conjuntos (split por `match_id`, con estratificación por competencia) y con exclusión explícita de los eventos de disparo del vector de entrada para garantizar una tarea de predicción causal.

El sistema es productizado a través de una API REST con FastAPI (`/v1/predict-possession`) y un frontend interactivo en React con visualización de trayectorias en campo y un medidor de probabilidad.

Índice

1. Introducción	2
1.1. El problema de negocio	2
1.2. Contexto técnico y motivación	2
1.3. Objetivo principal	2
1.4. Alcance del proyecto	3
1.5. Contribuciones	3
2. Dataset	3
2.1. StatsBomb Open Data	3
2.1.1. Campos disponibles por evento	4
2.2. Competencias seleccionadas	4
2.3. Estadísticas del dataset	4
3. Análisis Exploratorio de Datos (EDA)	5
3.1. Distribución de clases	5
3.2. Tasa de disparo por competencia	6
3.3. Longitud de secuencia	6
3.4. Progresión territorial	7
3.5. Análisis por patrón de juego	7
4. Metodología	8
4.1. Definición formal del problema	8
4.2. Pipeline de datos	8
4.3. Ingeniería de características	8
4.4. División de datos	9
4.5. Modelo baseline	9

4.6. Arquitectura GRU	10
5. Resultados	11
5.1. Curvas de entrenamiento	11
5.2. Métricas de evaluación	12
5.3. Curvas ROC y Precisión-Recall	13
5.4. Resultados por competencia	13
5.5. Calibración	14
6. API y Aplicación Web	14
6.1. Visión general del producto	14
6.2. Arquitectura técnica del sistema	15
6.3. Endpoints de la API	15
6.3.1. Predicción de una posesión (/v1/predict-possession)	15
6.3.2. Análisis completo de partido (/v1/analyze-match)	16
6.4. Frontend interactivo	17
6.4.1. Flujo de uso	17
6.4.2. Escenarios predefinidos	17
6.4.3. Componentes visuales	17
6.5. Cómo ejecutar el sistema	18
7. Discusión	18
7.1. Interpretación de los resultados	18
7.2. Limitaciones	19
7.3. Trabajo futuro	19
8. Conclusión	19
A. Estructura del repositorio	20
B. Instrucciones de ejecución	21
B.1. Requisitos	21
B.2. Pipeline completo	21
C. Glosario	21

Introducción

El problema de negocio

El fútbol moderno mueve miles de millones de dólares al año a través de fichajes, patrocinios, derechos televisivos y apuestas deportivas. En este contexto, **entender qué hace peligrosa una jugada** —antes de que ocurra el disparo— tiene valor directo para múltiples actores del ecosistema:

- **Cuerpos técnicos y analistas tácticos:** identificar qué tipo de posesiones genera riesgo real para el rival o para el propio equipo. Un entrenador puede revisar en segundos las 10 posesiones más peligrosas cedidas en el último partido sin necesidad de ver horas de video.
- **Scouts y departamentos de captación:** evaluar qué tan «amenazante» es el juego colectivo de un equipo más allá de estadísticas superficiales como porcentaje de posesión o número de remates.
- **Medios y plataformas de *broadcasting*:** enriquecer la retransmisión en vivo con una capa de inteligencia que señale jugadas de alto peligro antes de que terminen en gol, incrementando el engagement de la audiencia.
- **Plataformas de apuestas deportivas:** cuantificar la presión ofensiva de un equipo en tiempo real durante el partido como señal complementaria para mercados de gol siguiente / corners / disparos.

La métrica clave que responde a estas necesidades es:

$$P(\text{disparo} \mid \text{posesión})$$

es decir, la probabilidad de que una posesión ofensiva *termine en intento de gol*. A diferencia del *Expected Goals* (xG), que puntúa la calidad de un disparo ya ocurrido, esta métrica cuantifica la *peligrosidad de la construcción* incluso antes de que exista el disparo.

Contexto técnico y motivación

Las métricas tradicionales de fútbol (posesión %, número de disparos, pases completados) son estadísticas de resumen que pierden toda la información *secuencial*: la forma en que una jugada se construye, el orden de las acciones, cómo progresa el equipo hacia la portería. Un equipo puede tener el 60 % de la posesión con jugadas inocuas, mientras otro con el 40 % genera el doble de peligro real.

El modelado de posesiones como secuencias —usando redes neuronales recurrentes— permite capturar exactamente esa estructura temporal: quién pasó a quién, desde dónde y hacia dónde, bajo qué presión, durante cuánto tiempo.

Objetivo principal

Construir un sistema end-to-end que, dada la secuencia de eventos de una posesión, estime en tiempo real la probabilidad de que termine en disparo:

$$P(\text{disparo} \mid e_1, e_2, \dots, e_T) \quad (1)$$

donde e_t es el t -ésimo evento (pase, conducción, regate, etc.) y T es la longitud variable de la secuencia. El sistema expone esta predicción a través de una API REST y una interfaz visual interactiva.

Alcance del proyecto

- Fútbol masculino de clubes europeos con datos abiertos disponibles.
- Modelado a nivel de posesión — unidad de análisis = una posesión completa.
- Clasificación binaria: ¿termina esta posesión en disparo?
- Arquitectura GRU como modelo secuencial principal.
- API REST + frontend web como capa de producto.

Quedan fuera del alcance: visión computacional sobre video, modelos Transformer, competencias femeninas o internacionales, y clasificación de calidad del disparo (xG).

Contribuciones

1. Pipeline reproducible de descarga, parseo y etiquetado de posesiones.
2. Codificación de eventos en vectores de 40 características con gestión de secuencias de longitud variable.
3. Modelo GRU entrenado sin fuga de datos, con split estratificado por competencia.
4. Comparativa rigurosa contra baseline tabular (regresión logística).
5. API REST y frontend interactivo con visualización de campo listos para demo.

Dataset

StatsBomb Open Data

El dataset utilizado es **StatsBomb Open Data**, disponible públicamente en <https://github.com/statsbomb/open-data>. StatsBomb es una compañía de analítica deportiva que publica gratuitamente datos de eventos de partidos seleccionados para uso académico y de investigación.

Cada partido se representa como una lista de eventos JSON ordenados cronológicamente, donde cada evento describe una acción individual (pase, conducción, disparo, etc.) con metadatos espaciales y contextuales.

Campos disponibles por evento

Los campos relevantes para el modelado de posesiones incluyen:

Cuadro 1: Campos del esquema de eventos StatsBomb utilizados en este proyecto.

Campo	Descripción
possession	Identificador numérico de la posesión dentro del partido
type	Tipo de evento: <i>Pass</i> , <i>Carry</i> , <i>Dribble</i> , <i>Shot</i> , etc.
play_pattern	Patrón de inicio: <i>Regular Play</i> , <i>From Corner</i> , etc.
location	Coordenadas (x, y) en campo de 120×80 metros
pass_end_location	Coordenadas de destino del pase
carry_end_location	Coordenadas de destino de la conducción
duration	Duración del evento en segundos
under_pressure	Indicador booleano de presión defensiva
minute, second	Marca temporal en el partido

Competencias seleccionadas

Se seleccionaron cuatro competencias de clubes europeos masculinos con datos de eventos disponibles. La selección prioriza homogeneidad táctica (exclusión de torneos internacionales) y volumen de partidos.

Cuadro 2: Competencias incluidas en el dataset de entrenamiento.

Competencia	Temporada	ID comp.	ID temp.	Partidos
1. Bundesliga	2023/2024	9	281	34
La Liga	2020/2021	11	90	35
Ligue 1	2021/2022	7	108	26
Ligue 1	2022/2023	7	235	32
Total				127

Estadísticas del dataset

A partir de los 127 partidos se extrajeron 21,080 posesiones únicas. La distribución de clases refleja la naturaleza del fútbol: la mayoría de las posesiones no terminan en disparo.

Cuadro 3: Estadísticas generales del dataset de posesiones.

Métrica	Valor
Total de posesiones	21,080
Posesiones con disparo	3,024 (14.3 %)
Posesiones sin disparo	18,056 (85.7 %)
Razón de desequilibrio	$\approx 6 : 1$
Partidos	127
Promedio posesiones/partido	≈ 166

Análisis Exploratorio de Datos (EDA)

Distribución de clases

La figura 1 muestra la distribución de posesiones con y sin disparo. El desequilibrio de clases ($\approx 6 : 1$) es manejable para clasificación binaria estándar y fue tratado mediante pérdida BCE ponderada durante el entrenamiento del GRU (peso positivo = $18,056/3,024 \approx 5.97$).

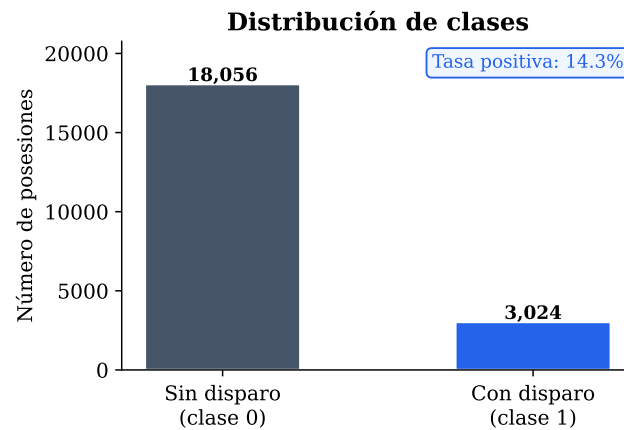


Figura 1: Distribución de clases en el dataset completo (21,080 posesiones).

Tasa de disparo por competencia

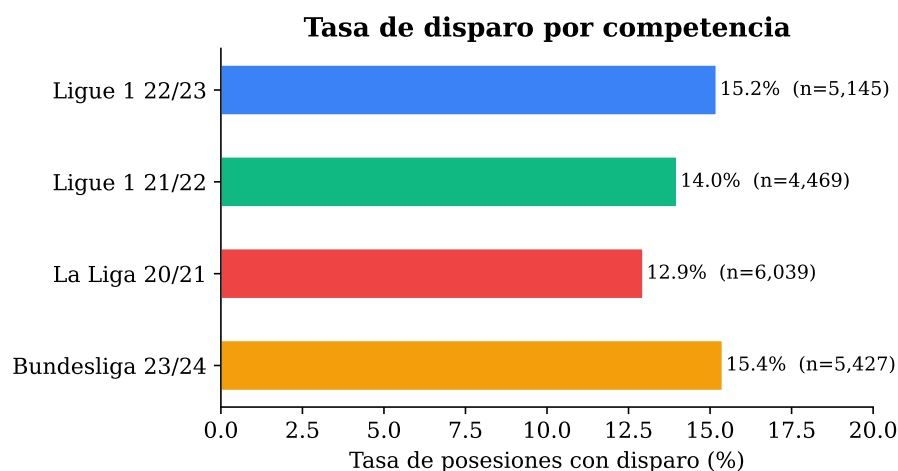


Figura 2: Porcentaje de posesiones que terminan en disparo por competencia.

Las cuatro ligas presentan tasas de disparo similares (entre 13 % y 16 %), lo que indica que el modelo no debería desarrollar sesgos fuertes por competencia derivados de esta variable.

Longitud de secuencia

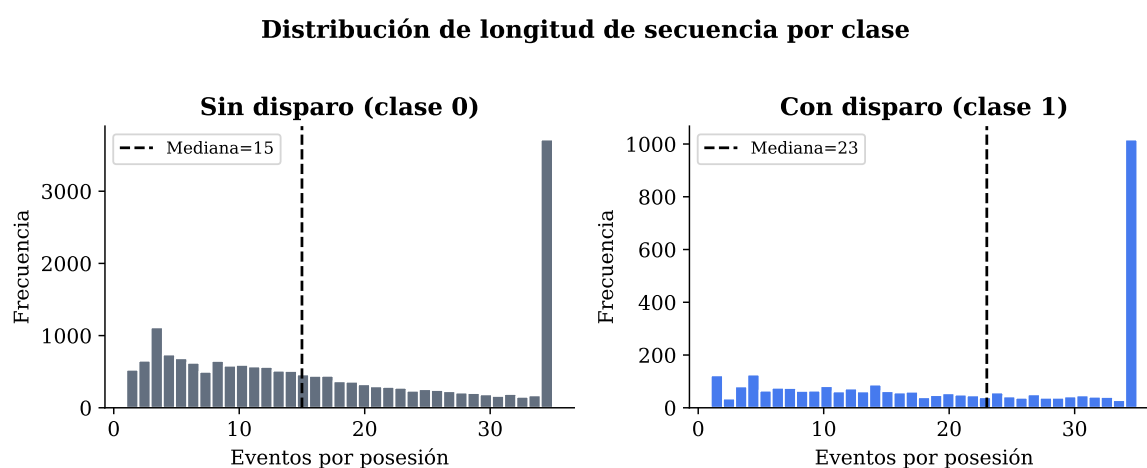


Figura 3: Distribución del número de eventos por posesión, separada por clase. Las posesiones con disparo tienden a ser más largas.

Las posesiones que terminan en disparo son notoriamente más largas que las que no: la diferencia en mediana refleja que los equipos construyen jugadas más elaboradas antes de llegar al disparo. Esta señal es aprovechable por un modelo secuencial.

Progresión territorial

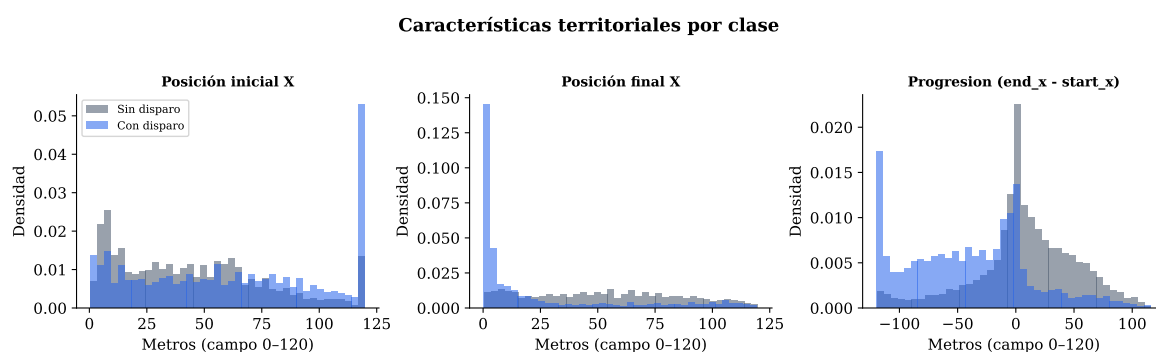


Figura 4: Distribución de características territoriales: posición inicial, posición final y progresión neta. Las posesiones con disparo muestran inicio y final más avanzados hacia la portería rival ($x = 120$).

La posición inicial y final de las posesiones con disparo están significativamente más adelantadas en el campo. La progresión neta también es mayor, aunque con mayor varianza (algunas posesiones progresan mucho desde zonas defensivas).

Análisis por patrón de juego

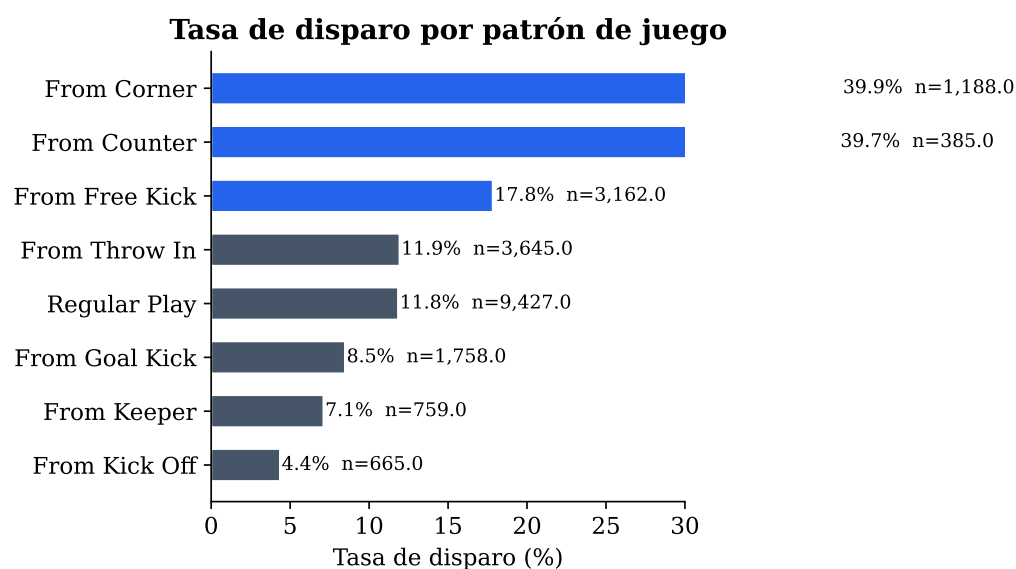


Figura 5: Tasa de disparo según el patrón de inicio de posesión. El tiro libre y el saque de esquina tienen tasas notablemente mayores.

Las jugadas a balón parado (tiro libre, saque de esquina) presentan las tasas de disparo más altas, lo que es coherente con el diseño táctica en fútbol moderno donde las jugadas ensayadas de este tipo frecuentemente finalizan en remate.

Metodología

Definición formal del problema

Sea $P_i = (e_1^{(i)}, e_2^{(i)}, \dots, e_{T_i}^{(i)})$ la secuencia de eventos de la i -ésima posesión, donde cada evento e_t es un vector de características $\mathbf{x}_t \in \mathbb{R}^{40}$. La etiqueta es:

$$y_i = \mathbb{I}[\text{la posesión } i \text{ contiene algún evento de tipo } Shot] \quad (2)$$

El objetivo es aprender una función $f_\theta : \mathbb{R}^{T \times 40} \rightarrow [0, 1]$ tal que:

$$\hat{y}_i = f_\theta(\mathbf{x}_1^{(i)}, \dots, \mathbf{x}_{T_i}^{(i)}) \approx P(\text{disparo} \mid P_i) \quad (3)$$

Nota crítica sobre fuga de datos: Los eventos de tipo *Shot* fueron *excluidos explícitamente* del vector de entrada. Incluirlos permitiría al modelo detectar trivialmente la presencia del disparo en la secuencia, invalidando la tarea de predicción. El modelo ve únicamente los eventos *previos* al disparo (o todos los eventos en posesiones sin disparo).

Pipeline de datos

El pipeline de procesamiento sigue los siguientes pasos:

1. **Descarga:** Se obtienen los archivos JSON de eventos para los 127 partidos seleccionados usando la librería `statsbombpy`.
2. **Agrupación por posesión:** Los eventos se agrupan por el campo `possession`, que es el identificador asignado por StatsBomb a cada posesión dentro de un partido.
3. **Filtrado:** Se eliminan eventos de tipo administrativo (*Starting XI*, *Half Start/End*, *Substitution*, *Tactical Shift*, *etc.*).
4. **Etiquetado:** Si algún evento de la posesión es de tipo *Shot*, la posesión recibe etiqueta $y = 1$.
5. **Codificación:** Cada evento se codifica en un vector de 40 dimensiones (descrito en la sección 4.3).
6. **Serialización:** Los datos procesados se guardan en formato Parquet para reproducibilidad.

Ingeniería de características

Cada evento es representado por un vector $\mathbf{x}_t \in \mathbb{R}^{40}$ compuesto de tres bloques:

Cuadro 4: Composición del vector de características por evento.

Bloque	Dimensiones	Descripción
Tipo de evento (one-hot)	23	Pass, Carry, Dribble, Ball Receipt*, etc.
Patrón de juego (one-hot)	8	Regular Play, From Corner, From Free Kick, etc.
Zona del campo (one-hot)	4	Defensiva ($x < 40$), Media, Atacante ($x > 80$), Descor
Características continuas	5	x_{norm} , y_{norm} , $x_{\text{end_norm}}$, $y_{\text{end_norm}}$, duration_norm + under_pressure (binario)
Total	40	

Las coordenadas se normalizan dividiendo por las dimensiones del campo (120×80 m). La duración se normaliza con un máximo de 10 s. Las secuencias se truncan o rellenan (*padding*) hasta una longitud máxima de $T_{\text{máx}} = 50$ eventos.

División de datos

El dataset se divide a nivel de partido (**match_id**), garantizando que todas las posesiones de un mismo partido pertenezcan al mismo conjunto. La estratificación por competencia asegura representación de las cuatro ligas en cada split.

Cuadro 5: Distribución de partidos y posesiones por split.

Split	Ratio	Partidos	Posesiones	Disparos	Tasa
Entrenamiento	70 %	88	14,582	2,114	14.5 %
Validación	15 %	19	3,192	457	14.3 %
Prueba	15 %	20	3,306	453	13.7 %
Total		127	21,080	3,024	14.3 %

Reglas de split:

- División por **match_id** — ninguna posesión cruza entre splits.
- Estratificación por **competition_label** — cada split contiene representación de las cuatro competencias.
- Semilla fija (**random_state=42**) para reproducibilidad.

Modelo baseline

El baseline es una **regresión logística** entrenada sobre características *agregadas* a nivel de posesión (sin información secuencial). El preprocesamiento incluye normalización estándar para variables numéricas y codificación one-hot para variables categóricas.

Cuadro 6: Características del modelo baseline (regresión logística).

Tipo	Características
Numéricas (11)	n_eventos, n_pases, n_conducciones, n_regates, n_presiones, n_acciones_tercio_atacante, start_x, end_x, progresión, duración_total, minuto_inicio
Categorías (3)	zona_inicio, zona_final, patrón_de_juego

Arquitectura GRU

El modelo principal es una **GRU (Gated Recurrent Unit)** de dos capas. La elección de GRU sobre LSTM se justifica por su eficiencia computacional comparable con rendimiento similar en secuencias de longitud moderada. Se descartó Transformer en esta versión dado el tamaño del dataset (insuficiente para aprender atención multi-cabeza de manera estable sin preentrenamiento).

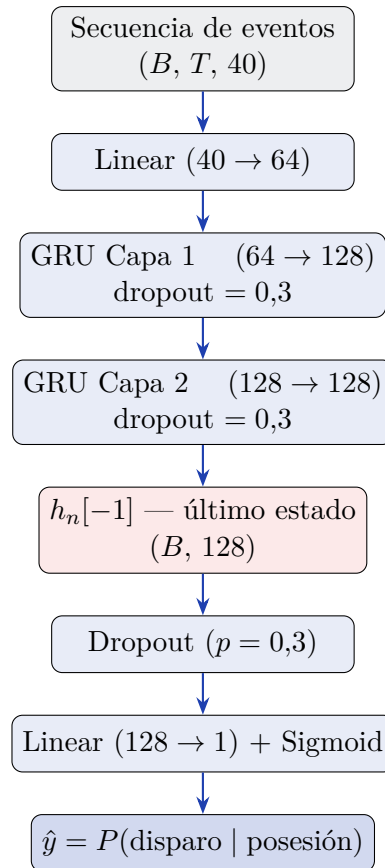


Figura 6: Arquitectura del modelo GRU para predicción de disparo en posesión.

Cuadro 7: Hiperparámetros del modelo GRU.

Hiperparámetro	Valor
Tamaño de entrada	40
Proyección	64
Celdas ocultas	128
Capas GRU	2
Dropout	0.3
Parámetros totales	176,385
Función de pérdida	BCE ponderada ($w_+ \approx 5,97$)
Optimizador	AdamW ($\alpha = 3 \times 10^{-4}$, $\lambda = 10^{-4}$)
Scheduler	CosineAnnealingLR
Épocas máximas	40
Paciencia (early stop)	8 épocas (por PR-AUC en validación)
Tamaño de batch	256
Longitud máx. secuencia	50

Gestión de longitud variable: Se usa `pack_padded_sequence` de PyTorch para evitar que el padding influya en la actualización de los estados ocultos. El estado final $h_n[-1]$ (última capa, último paso temporal real) se usa como representación de la posesión completa.

Entorno de entrenamiento: GPU NVIDIA GeForce RTX 5080 (16 GB VRAM), PyTorch 2.10.0 con soporte CUDA 13.0.

Resultados

Curvas de entrenamiento

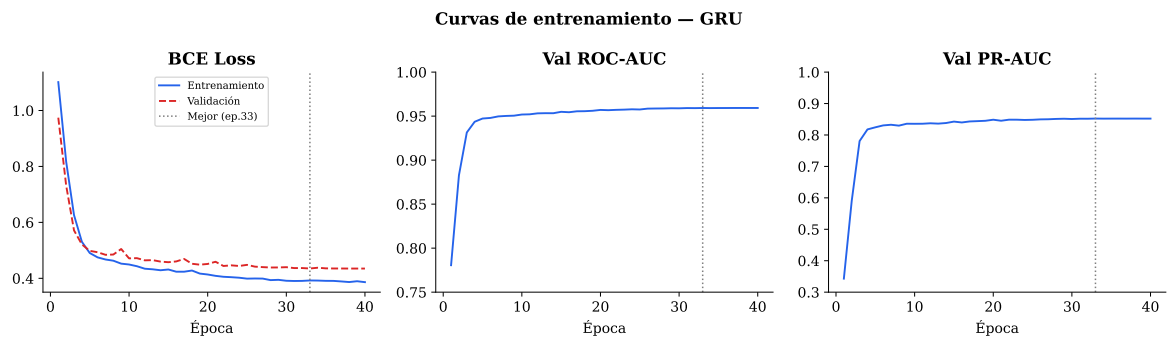


Figura 7: Evolución de la pérdida BCE, ROC-AUC y PR-AUC en validación durante el entrenamiento. La línea punteada vertical indica la época del mejor checkpoint (seleccionado por PR-AUC en validación).

El modelo converge establemente a lo largo de las 40 épocas. La pérdida de validación no diverge respecto a la de entrenamiento, indicando ausencia de sobreajuste severo. El

PR-AUC en validación alcanza un plateau a partir de la época 20 en torno a 0,85.

Métricas de evaluación

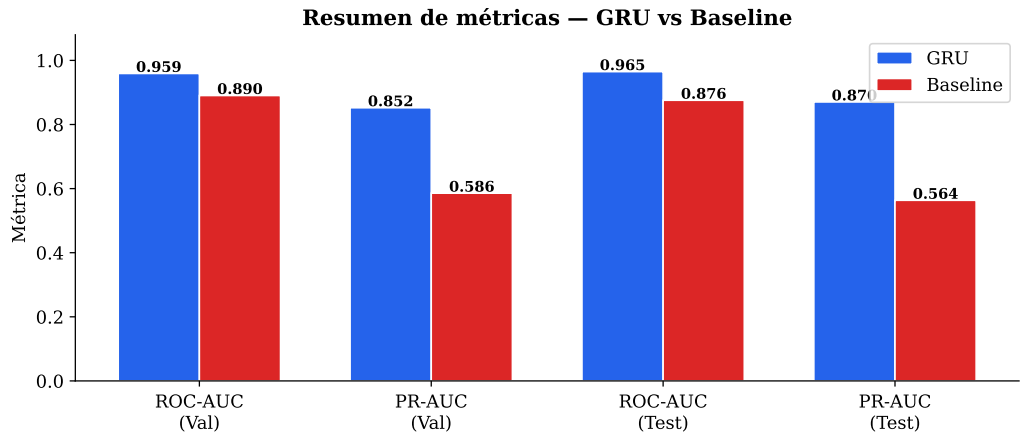


Figura 8: Comparativa de métricas principales entre GRU y baseline de regresión logística, en validación y prueba.

Cuadro 8: Resultados completos de evaluación — GRU vs. Baseline.

Modelo	Split	ROC-AUC	PR-AUC	Log-Loss	Brier
GRU	Validación	0.9592	0.8522	0.2515	0.0759
	Prueba	0.9649	0.8703	0.2310	0.0702
Baseline (LR)	Validación	0.8904	0.5857	0.4515	0.1438
	Prueba	0.8756	0.5635	0.4451	0.1409
Mejora GRU vs Baseline (Test PR-AUC)			+30.7 pp		

El PR-AUC es la métrica principal por ser más informativa que el ROC-AUC en presencia de desequilibrio de clases. Un modelo aleatorio obtendría PR-AUC $\approx$ 0,143 (igual a la tasa base); el GRU alcanza 0,870 en prueba, superando al baseline en más del doble.

Curvas ROC y Precisión-Recall

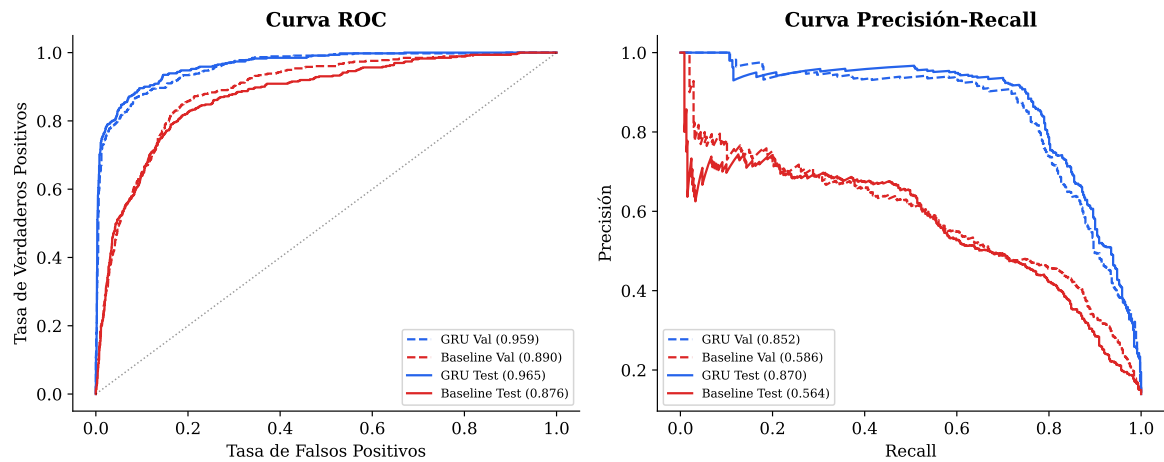


Figura 9: Curvas ROC (izquierda) y Precisión-Recall (derecha) para GRU y baseline, en validación (línea discontinua) y prueba (línea sólida).

Las curvas muestran que el GRU domina al baseline en ambas métricas y en ambos conjuntos. La consistencia entre validación y prueba confirma buena generalización.

Resultados por competencia

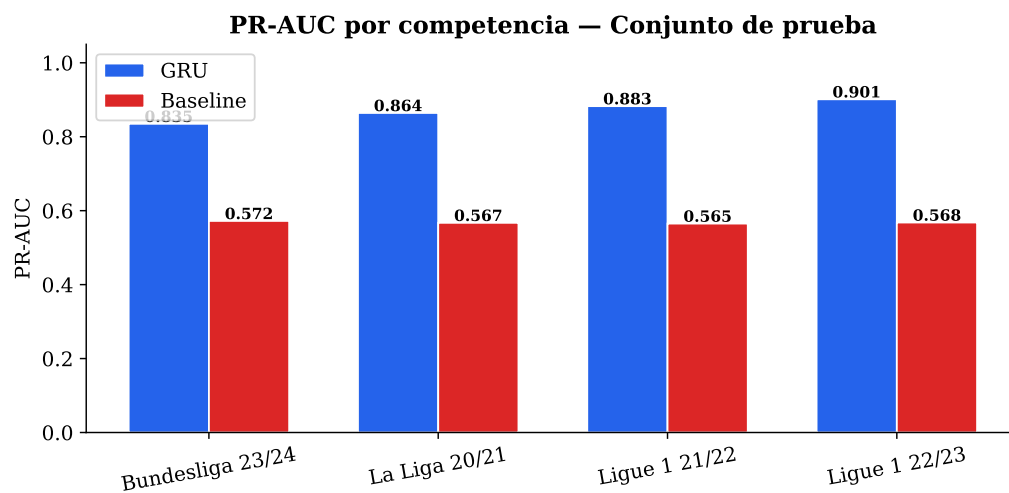


Figura 10: PR-AUC por competencia en el conjunto de prueba. El GRU supera al baseline en todas las ligas sin excepción.

Cuadro 9: Resultados por competencia en el conjunto de prueba.

Competencia	GRU		Baseline	
	ROC-AUC	PR-AUC	ROC-AUC	PR-AUC
Bundesliga 2023/24	0.9564	0.8350	0.8840	0.5723
La Liga 2020/21	0.9652	0.8643	0.8861	0.5672
Ligue 1 2021/22	0.9643	0.8828	0.8639	0.5651
Ligue 1 2022/23	0.9728	0.9013	0.8684	0.5680
Total	0.9649	0.8703	0.8756	0.5635

El modelo generaliza de forma consistente entre ligas, con PR-AUC entre 0.835 y 0.901. La Ligue 1 2022/23 presenta el mejor desempeño, posiblemente relacionado con el mayor volumen de posesiones disponibles en ese conjunto de prueba.

Calibración

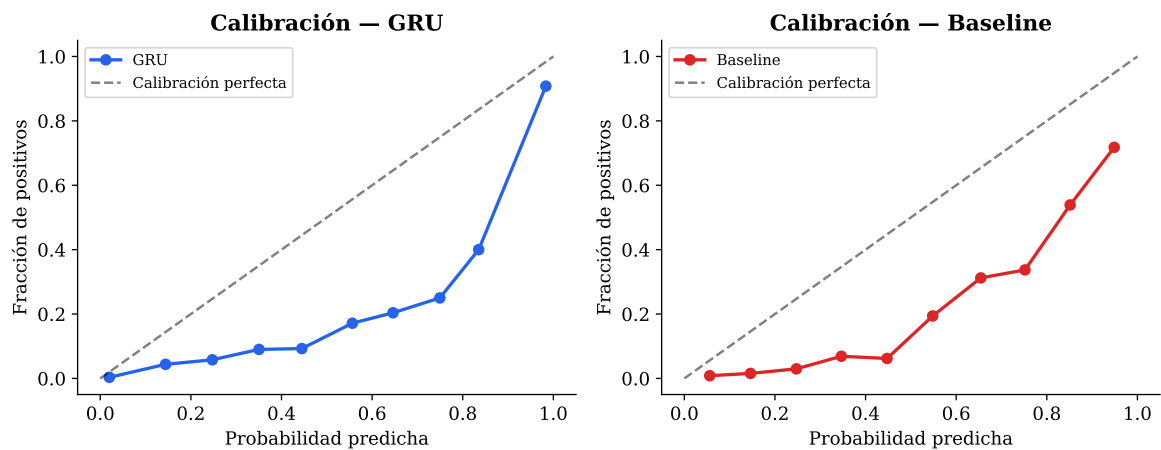


Figura 11: Curvas de calibración para GRU (izquierda) y baseline (derecha) en el conjunto de validación. El eje diagonal representa calibración perfecta.

El GRU muestra mejor calibración que el baseline: sus probabilidades predichas se aproximan más a la calibración perfecta (diagonal), aunque con ligera sobreconfianza en el rango $[0.3, 0.7]$. El baseline sobreestima sistemáticamente en todos los rangos. Una isotonic regression o Platt scaling podría mejorar la calibración del GRU en producción.

API y Aplicación Web

Visión general del producto

El sistema va más allá del modelo estadístico: el objetivo es que cualquier analista, periodista o desarrollador pueda **consumir las predicciones sin necesidad de**

conocimiento técnico en ML. Para ello, el modelo entrenado se empaqueta en dos capas:

1. **API REST** — interfaz programática que acepta eventos de posesión en JSON y devuelve la probabilidad de disparo. Consumible desde cualquier lenguaje o plataforma.
2. **Frontend web** — aplicación React con visualización interactiva del campo, medidor de peligrosidad y línea de tiempo de eventos.

Arquitectura técnica del sistema

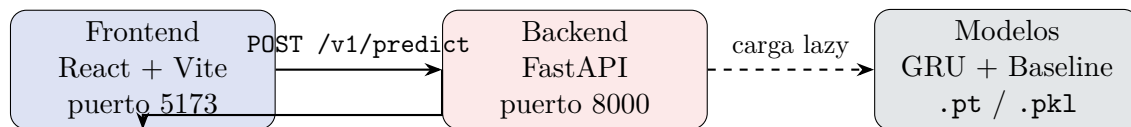


Figura 12: Componentes del sistema. El frontend consume la API REST; los modelos se cargan en memoria al primer request (*lazy loading*).

Stack tecnológico:

- **Backend:** Python 3.12, FastAPI, Uvicorn, PyTorch, scikit-learn. CORS habilitado para comunicación con el frontend local. Documentación automática disponible en `/docs` (Swagger UI).
- **Frontend:** React 18, TypeScript, Vite, Tailwind CSS. Visualización de campo con SVG nativo (escala $120 \times 80 \text{ m} \rightarrow 600 \times 400 \text{ px}$). Sin dependencias de mapas externos.

Endpoints de la API

Cuadro 10: Endpoints expuestos por el backend FastAPI.

Método	Ruta	Descripción
GET	<code>/health</code>	Estado del servicio y modelos cargados
GET	<code>/v1/models</code>	Lista de modelos disponibles con metadatos
POST	<code>/v1/predict-possession</code>	Probabilidad de disparo para una posesión
POST	<code>/v1/analyze-match</code>	Ranqueo de todas las posesiones de un partido

Predicción de una posesión (`/v1/predict-possession`)

El endpoint principal acepta una lista de eventos en formato JSON y retorna la probabilidad predicha. Se puede especificar qué modelo usar: "gru" o "baseline".

```

1 POST /v1/predict-possession
2 {
3   "model": "gru",
4   "events": [
  
```

```
5     {"event_type": "Pass",      "play_pattern": "Regular_Play",
6       "x": 75.0, "y": 40.0,    "end_x": 88.0, "end_y": 35.0,
7       "duration": 0.8,        "under_pressure": 0},
8     {"event_type": "Carry",    "play_pattern": "Regular_Play",
9       "x": 88.0, "y": 35.0,    "end_x": 105.0, "end_y": 38.0,
10      "duration": 1.4,         "under_pressure": 1},
11     {"event_type": "Dribble",  "play_pattern": "Regular_Play",
12       "x": 105.0, "y": 38.0,   "end_x": null, "end_y": null,
13       "duration": 0.5,        "under_pressure": 1}
14   ]
15 }
```

Listing 1: Request de predicción — ataque en campo rival bajo presión.

```
1 {
2   "shot_probability": 0.6821,
3   "model_used": "gru",
4   "n_events": 3,
5   "model_version": "1.0.0",
6   "timestamp": "2026-06-23T17:45:12+00:00"
7 }
```

Listing 2: Respuesta — probabilidad de disparo y metadatos.

Un valor de 0,68 indica que posesiones con este patrón terminan en disparo en $\approx 68\%$ de los casos según lo aprendido por el modelo.

Análisis completo de partido (/v1/analyze-match)

Este endpoint recibe todas las posesiones de un partido y devuelve un ranking ordenado de mayor a menor peligrosidad. Útil para generar resúmenes post-partido automáticos sin revisión manual de video.

```
1 {
2   "possessions_ranked": [
3     {"possession_id": 47, "shot_probability": 0.891, "n_events":
4       12,
5       "zone": "attacking", "play_pattern": "From_Corner"},
6     {"possession_id": 83, "shot_probability": 0.834, "n_events": 8,
7       "zone": "attacking", "play_pattern": "Regular_Play"},
8     ...
9   ],
10  "match_summary": {
11    "total_possessions": 162,
12    "high_danger": 18,
13    "medium_danger": 41,
14    "low_danger": 103
15  }
16 }
```

Listing 3: Respuesta de analyze-match (extracto).

Frontend interactivo

Flujo de uso

El frontend está diseñado para ser usado por analistas sin conocimientos de programación. El flujo típico es:

1. **Seleccionar escenario:** el usuario elige uno de los tres escenarios de posesión predefinidos (o puede enviar su propia secuencia de eventos vía la API directamente).
2. **Elegir modelo:** selecciona entre el GRU (mayor precisión) y el baseline de regresión logística (más interpretable, sin GPU).
3. **Enviar a la API:** con un clic, el frontend hace un POST al backend y recibe la probabilidad en milisegundos.
4. **Interpretar:** el resultado se muestra en tres componentes visuales simultáneos: campo, gauge y línea de tiempo.

Escenarios predefinidos

Cuadro 11: Escenarios de posesión predefinidos en el frontend.

Escenario	Descripción táctica	$\hat{P}$ esperada
Ataque Peligroso	Pase en campo propio $\rightarrow$ conducción progresiva $\rightarrow$ regate en área rival. Alta penetración, presión defensiva intensa en la zona final.	$> 0,60$
Construcción Segura	Pases cortos en campo propio y zona media. Sin progresión hacia portería, sin presión. Posesión de mantenimiento sin intención ofensiva clara.	$< 0,15$
Contraataque	Recuperación de balón $\rightarrow$ conducción veloz en campo propio $\rightarrow$ pase largo al espacio. Transición rápida sin construcción pausada.	$0,30-0,55$

Componentes visuales

Campo SVG con trayectorias: Renderizado a escala real del campo (120×80 m) en SVG. Cada evento de la posesión se dibuja como un segmento de línea coloreado según el tipo de acción (pase en azul, conducción en naranja, regate en verde). Los círculos marcan el inicio y fin de cada acción.

Medidor de probabilidad (gauge): Semicírculo animado que va de verde ($P < 0,3$, peligro bajo) a amarillo ($0,3 \leq P < 0,6$, peligro medio) a rojo ($P \geq 0,6$, peligro alto). El número de probabilidad se muestra en el centro con tamaño de fuente proporcional al riesgo.

Línea de tiempo de eventos: Lista horizontal de chips con el tipo y coordenadas de cada evento de la posesión, con scroll horizontal para secuencias largas. Permite al analista seguir la jugada evento a evento.



[INSERTAR CAPTURA] Frontend completo — escenario “Ataque Peligroso”: campo con trayectoria, gauge en rojo (>60 %), línea de tiempo de eventos



[INSERTAR CAPTURA] Frontend — escenario “Construcción Segura” (gauge verde) vs. “Ataque Peligroso” (gauge rojo) lado a lado, o alternando modelos GRU vs. Baseline para la misma posesión



[INSERTAR CAPTURA] Swagger UI en <http://localhost:8000/docs> mostrando los endpoints disponibles y el esquema JSON del request

Cómo ejecutar el sistema

```

1 # 1. Entrenar los modelos (requiere GPU; usa el venv con CUDA)
2 python run_pipeline.py all
3
4 # 2. Levantar el backend (otra terminal)
5 cd MCD-DeepLearning-FinalProject
6 uvicorn backend.app.main:app --reload --port 8000
7
8 # 3. Levantar el frontend (otra terminal)
9 cd frontend
10 npm install
11 npm run dev
12 # Abrir http://localhost:5173 en el navegador
13
14 # La documentacion automatica de la API estara en:
15 # http://localhost:8000/docs

```

Listing 4: Pasos para levantar el sistema completo.

Discusión

Interpretación de los resultados

El modelo GRU supera ampliamente al baseline tabular en todas las métricas y ligas. Esta brecha de +30,7 pp en PR-AUC no es trivial: demuestra que la *secuencia* de eventos contiene información predictiva que no está capturada por los agregados. Aspectos como

el orden de las acciones, la progresión incremental hacia la portería o la acumulación de presión defensiva son patrones que el GRU puede capturar y el baseline no.

Los resultados son estables entre validación y prueba (PR-AUC de 0,852 y 0,870 respectivamente), lo que sugiere buena generalización dentro del dominio de los clubes europeos analizados.

Limitaciones

1. **Representatividad del dataset:** 127 partidos de cuatro competencias es un dataset relativamente pequeño para deep learning. Modelos de producción como los de StatsBomb o Google DeepMind utilizan miles de partidos.
2. **Vista post-hoc de la posesión:** El modelo ve la posesión completa (hasta su terminación), no el estado parcial en tiempo real. Para predicción en vivo se requeriría una evaluación adicional con secuencias truncadas.
3. **Ausencia de contexto espacial 360°:** Los datos de *freeze frame* (posición de todos los jugadores en el instante de cada evento) están disponibles en StatsBomb para estas competencias pero no fueron integrados en esta versión.
4. **Calibración imperfecta:** El GRU muestra sobreconfianza moderada en el rango medio de probabilidades. Para aplicaciones donde la probabilidad calibrada es crítica, se recomienda post-calibración con Platt scaling.
5. **Dominio acotado:** El modelo no fue evaluado en torneos internacionales ni competencias femeninas; su transferibilidad a esos dominios es desconocida.

Trabajo futuro

- Integrar datos de freeze frame (360°) para añadir contexto espacial.
- Explorar una formulación en tiempo real (predicción tras cada evento).
- Ampliar el dataset a más temporadas y ligas.
- Añadir la tarea secundaria $P(\text{gol} \mid \text{posesión})$.
- Evaluar arquitecturas Transformer con datasets más grandes.

Conclusión

Este proyecto demuestra que el modelado secuencial de posesiones de fútbol con redes recurrentes produce predicciones significativamente superiores a los enfoques basados en características agregadas.

El modelo GRU desarrollado alcanza un **PR-AUC de 0.870 en el conjunto de prueba** con 21,080 posesiones extraídas de 127 partidos europeos, superando en +30,7 pp al baseline tabular. La pipeline completa — desde la descarga de datos hasta la API de inferencia — es reproducible y está documentada.

El sistema está listo para ser demostrado como aplicación funcional: la API FastAPI acepta secuencias de eventos y retorna probabilidades de disparo en tiempo real, mientras que el frontend React proporciona una interfaz visual para análisis táctico.

Desde la perspectiva del aprendizaje profundo, el proyecto valida la elección de GRU sobre Transformer para este tamaño de dataset, e ilustra la importancia de un diseño experimental riguroso (split sin fuga, exclusión de señales triviales, métricas apropiadas para clases desbalanceadas).

Estructura del repositorio

```

1 MCD-DeepLearning-FinalProject/
2 |-- run_pipeline.py           # Script maestro (todas las etapas
3 |                             )
4 |-- src/
5 |   |-- data/
6 |     |-- download_statsbomb.py # Descarga datos open de
7 |       StatsBomb
8 |     |-- build_possession_dataset.py # Parser eventos ->
9 |       posesiones
10 |    |-- create_splits.py         # Split estratificado por
11 |      match_id
12 |    |-- features/
13 |      |-- encode_possessions.py  # Codificación de eventos
14 |        (40 dims)
15 |    |-- models/
16 |      |-- lstm_model.py          # Arquitectura GRU
17 |      |-- dataset.py             # PyTorch Dataset/
18 |        DataLoader
19 |      |-- train_baseline.py      # Entrenamiento logistic
20 |        regression
21 |      |-- train_lstm.py          # Loop de entrenamiento GRU
22 |      |-- evaluation/
23 |        |-- evaluate_lstm.py     # Evaluación final GRU vs
24 |          baseline
25 |-- backend/
26 |   |-- app/main.py              # FastAPI - endpoints de
27 |     inferencia
28 |   |-- requirements.txt
29 |-- frontend/
30 |   |-- src/App.tsx              # React frontend con
31 |     visualización
32 |-- data/
33 |   |-- raw/statsbomb/           # JSONs descargados
34 |   |-- processed/
35 |     |-- possessions/possessions.parquet
36 |     |-- splits/{train,validation,test}_matches.csv
37 |-- models/trained/
38 |   |-- gru_best.pt               # Mejor checkpoint GRU
39 |   |-- baseline_logreg.pkl       # Modelo baseline
40 |     serializado
41 |   |-- gru_train_log.json        # Métricas por época
42 |   |-- gru_eval_results.json     # Evaluación final completa
43 |-- notebooks/
44 |   |-- eda/01_statsbomb_scope_validation.ipynb

```

```
34 | | -- eda/02_possession_dataset_eda.ipynb
35 | | -- experiments/01_baseline_results.ipynb
36 | | -- experiments/02_gru_results.ipynb
37 | -- report/
38 | | -- main.tex # Este documento
39 | | -- generate_figures.py # Genera figuras con
    matplotlib
40 | -- figures/ # PDFs y PNGs de figuras
```

Listing 5: Estructura del repositorio del proyecto.

Instrucciones de ejecución

Requisitos

- Python 3.12+
- PyTorch con soporte CUDA (recomendado para entrenamiento)
- Node.js 22+ (para el frontend)
- Dependencias Python: `pandas`, `scikit-learn`, `statsbombpy`, `torch`, `fastapi`, `uvicorn`, `pyarrow`, `tqdm`, `joblib`, `matplotlib`

Pipeline completo

```
1 # Pipeline de datos y entrenamiento (requiere Python con CUDA)
2 python run_pipeline.py all
3
4 # Backend (API REST)
5 uvicorn backend.app.main:app --reload --port 8000
6
7 # Frontend (otra terminal)
8 cd frontend && npm install && npm run dev
9 # Abrir http://localhost:5173
```

Listing 6: Ejecución del pipeline completo.

Glosario

Posesión

Secuencia continua de eventos realizados por el mismo equipo, identificada por el campo `possession` en los datos de StatsBomb.

GRU

Gated Recurrent Unit. Variante simplificada de LSTM con dos puertas (actualización y reinicio) en lugar de tres.

PR-AUC

Área bajo la curva Precisión-Recall. Métrica robusta para datasets con desequilibrio de clases; un clasificador aleatorio obtiene $\text{PR-AUC} \approx \text{tasa base}$.

ROC-AUC

Área bajo la curva Receiver Operating Characteristic. Mide la capacidad discriminativa del modelo independientemente del umbral de decisión.

BCE

Binary Cross-Entropy. Función de pérdida estándar para clasificación binaria:
 $\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.

Pack padded sequence

Técnica de PyTorch que comprime las secuencias eliminando el padding durante el procesamiento RNN para mayor eficiencia.

Freeze frame

Dato de StatsBomb 360° que captura la posición de todos los jugadores visibles en el campo en el momento de un evento.

xG

Expected Goals. Probabilidad de que un disparo específico resulte en gol, basada en características del disparo. Distinto de $P(\text{disparo} \mid \text{posesión})$ que es lo que este proyecto predice.